

Module 2: Privileged Containers & Attack Vectors — Assessment Questions

Module-Level Assessment (3 MCQs)

Question 1: Privileged Container Behavior

A developer argues that running a container with `--privileged` is acceptable because “the application inside the container is trusted and only performs file processing.” As a security engineer, which response BEST explains the risk?

- A) Privileged containers only affect networking and are safe for file processing workloads
- B) The `--privileged` flag disables Seccomp, AppArmor, and capability restrictions, and grants access to all host devices — meaning any vulnerability in the application, its dependencies, or the base image could be exploited to achieve full host compromise without needing any additional misconfiguration
- C) Privileged containers are acceptable if the application is trusted, as the trust boundary prevents exploitation
- D) The `--privileged` flag only adds extra Linux capabilities but maintains Seccomp and AppArmor protections, so the risk is moderate

Correct Answer: B

Explanation: The `--privileged` flag simultaneously disables multiple security layers: it grants all 41 Linux capabilities (not just the default 14), disables Seccomp system call filtering, disables AppArmor/SELinux mandatory access control, and provides access to all host devices. The “trusted application” argument fails because the application is not the only code running — the container also contains the base image’s OS packages, all application dependencies, and any libraries. A vulnerability in any of these components — combined with the complete absence of security controls — provides a direct path to host compromise. Trust in the application does not extend to its entire dependency chain.

Question 2: Docker Socket Escape

During a penetration test, you gain shell access to a container and discover that `/var/run/docker.sock` is mounted inside the container. The container is running with default (non-privileged) settings. What is the MOST effective attack path?

- A) Use the Docker socket to list running containers, then attempt to exploit a kernel vulnerability to escape
- B) Use the Docker socket to create a new privileged container with the host’s root filesystem mounted, then chroot into the host filesystem to gain root shell access on the host
- C) Read the Docker socket file to extract Docker daemon credentials, then use those credentials to SSH to the host
- D) The Docker socket cannot be used for privilege escalation from a non-privileged container because it requires `CAP_SYS_ADMIN`

Correct Answer: B

Explanation: The Docker socket provides full access to the Docker API, which runs as root on the host. Through this API, an attacker can create any container with any configuration — including `--privileged` with a host filesystem bind mount (`-v /:/hostfs`). No special capabilities are needed in the current container; the escalation happens through the new container. The attacker creates a new privileged container, mounts the host's root filesystem, and uses `chroot` to get a full root shell on the host. This is a well-documented, trivial attack that requires no exploit development — only standard Docker commands. The Docker socket does not contain extractable credentials (C), and it does not require `SYS_ADMIN` to use (D).

Question 3: CVE Impact Assessment

Your organization discovers that all production hosts are running a Linux kernel version affected by CVE-2022-0185 (heap overflow in filesystem context handling, exploitable with `CAP_SYS_ADMIN`). Your containers run with default Docker settings (no `--privileged`, no extra capabilities). An operations engineer suggests that since no containers have `CAP_SYS_ADMIN`, the hosts are not at risk. Is this assessment correct?

- A)** Yes — without `CAP_SYS_ADMIN` in any container, the vulnerability cannot be exploited, so patching can be deferred
- B)** Partially — the direct exploit requires `CAP_SYS_ADMIN`, but if unprivileged user namespaces are enabled on the host, any container process can create a user namespace and obtain `CAP_SYS_ADMIN` within it, potentially triggering the vulnerability. The kernel must still be patched
- C)** No — kernel vulnerabilities affect all processes regardless of capabilities, so the assessment is completely wrong
- D)** Yes — Docker's default Seccomp profile blocks the `fsconfig` system call, so the exploit is prevented

Correct Answer: B

Explanation: The assessment is partially correct but dangerously incomplete. While CVE-2022-0185 requires `CAP_SYS_ADMIN`, the capability can be obtained in the initial user namespace on systems where `kernel.unprivileged_userns_clone` is enabled (which is the default on many distributions including Ubuntu). A container process can create a user namespace and obtain `CAP_SYS_ADMIN` within it, which may be sufficient to trigger the vulnerability depending on the kernel version and configuration. Docker's default Seccomp profile does NOT block the `fsconfig` system call (D is incorrect). Option C overstates the case — capabilities do matter for exploitation, but the user namespace vector must be considered. The correct action is to patch the kernel AND consider disabling unprivileged user namespaces as an additional mitigation.

Final Assessment Contribution (2 Comprehensive MCQs)

Final Question 1: Multi-Vector Attack Scenario

You are investigating a security incident on a production host running 8 containers. The host runs Docker with default settings. One container — a CI/CD build agent — has the Docker socket mounted (`-v /var/run/docker.sock:/var/run/docker.sock`). The investigation reveals:

1. The CI/CD container was compromised through a malicious build dependency
2. A new container appeared in `docker ps` with `--privileged` and `-v /:/hostfs`

3. A new SSH key was added to `/root/.ssh/authorized_keys` on the host
4. A cron job was installed at `/etc/cron.d/update-check` running a reverse shell

Which assessment BEST describes the attack chain?

- A)** The attacker exploited a kernel vulnerability to escape the CI/CD container, then created the privileged container as a persistence mechanism
- B)** The attacker used the mounted Docker socket to create a new privileged container with host filesystem access, then used that container to add an SSH key and cron-based reverse shell for persistent host access
- C)** The attacker compromised the Docker daemon remotely through an exposed TCP port, then created containers and modified host files
- D)** The attacker exploited CVE-2019-5736 to overwrite the runc binary, which then created the privileged container automatically

Correct Answer: B

Explanation: The evidence perfectly matches a Docker socket escalation chain: (1) the CI/CD container was compromised through a supply chain attack (malicious dependency), (2) the attacker used the mounted Docker socket — which provides full Docker API access — to create a new privileged container with the host's root filesystem mounted at `/hostfs`, (3) from the privileged container, the attacker wrote an SSH key to the host's root `authorized_keys` file for persistent remote access, and (4) installed a cron-based reverse shell for redundant persistence. This requires no kernel exploit (A), no exposed TCP port (C), and no runtime vulnerability (D). It's entirely enabled by the single misconfiguration of mounting the Docker socket in the CI/CD container. The attack complexity is very low — only standard Docker commands are needed.

Final Question 2: Defensive Architecture

After reviewing the attacks covered in Module 2, your team is tasked with hardening a container deployment. Which combination of controls would MOST effectively prevent the demonstrated attack vectors while maintaining operational functionality?

- A)** Deploy all containers with `--privileged` but add strict AppArmor profiles to limit application behavior, and use Docker Content Trust for image verification
- B)** Use `--cap-drop=ALL` on all containers, replace Docker socket mounts with a least-privilege API proxy (e.g., Docker Socket Proxy), enforce Seccomp profiles, run containers as non-root, use read-only root filesystems, and deploy admission controllers to block privileged containers
- C)** Migrate all workloads to virtual machines to eliminate container escape risks entirely, and run Docker inside each VM for container management
- D)** Enable user namespaces globally, disable all network access for containers, and only allow containers to run pre-approved images from a private registry

Correct Answer: B

Explanation: Option B implements defense-in-depth against all demonstrated attack vectors: `--cap-drop=ALL` prevents capability-based escapes (CAP_SYS_ADMIN escape from Video 2.3); replacing Docker socket mounts with a proxy prevents socket-based escalation (Video 2.3); Seccomp profiles reduce the kernel attack surface (relevant to CVE-2022-0185 from Video 2.4); non-root containers reduce the impact of application compromise; read-only filesystems prevent persistence mechanisms; and admission controllers (OPA Gatekeeper, Kyverno) enforce these policies cluster-wide, preventing developers

from deploying privileged containers. Option A is contradictory — `--privileged` disables AppArmor, making the profiles ineffective. Option C is impractical and doesn't address the actual misconfiguration risks. Option D is operationally infeasible — disabling all network access would make most containerized applications non-functional.